[ETHICAL HACKING INTERNSHIP · MARCH 2026]

IDS Evasion Using Fragmented Ping

Snort 2.9.20 IDS | ICMP Fragmentation | hping3 Bypass | dsize:>7 Threshold

✓ IDS EVASION SUCCESSFUL

ZERO alerts during fragmented ping · 5/5 packets delivered

IDS	Snort 2.9.20 GRE (Build 82) on Ubuntu 24.04.4 LTS
ATTACK	hping3 -1 -f -d 1 (1-byte payload, forced fragmentation)
ATTACKER	Kali Linux — 192.168.109.129 (VMnet8/NAT)
DEFENDER	Ubuntu 24.04.4 LTS — 192.168.109.132 (VMnet8/NAT)
RULE	alert icmp ... dsize:>7 → SID:1000001
DATE	09 March 2026

Prepared by **Sai Aditya** · Ethical Hacking Intern · March 2026

Assessment 11C

1. Overview — IDS Evasion Concepts

An Intrusion Detection System (IDS) monitors network traffic and generates alerts when suspicious patterns are detected. **Snort** is the industry-standard open-source IDS that uses signature-based rules to identify threats. In this assessment, Snort was deployed on Ubuntu 24.04 LTS to detect ICMP ping traffic from a Kali Linux attacker on the same VMnet8 subnet.

IP fragmentation is a network mechanism that splits large packets into smaller pieces for transmission. Attackers exploit fragmentation to bypass IDS detection — by sending ICMP packets with extremely small payloads using hping3, the data size falls below the Snort rule's **dsize:>7** detection threshold, causing the rule to never fire.

Attacker VM	Kali Linux — 192.168.109.129 (VMnet8/NAT)
Defender VM	Ubuntu 24.04.4 LTS — 192.168.109.132 (VMnet8/NAT)
IDS Software	Snort 2.9.20 GRE (Build 82) — listening on ens33
Evasion Tool	hping3 — ICMP mode with -f (fragmentation) and -d 1 (1-byte payload)
Network	VMware VMnet8 (NAT) — both VMs on 192.168.109.0/24

2. Phase 1 — VM Setup & Network Configuration

Ubuntu Server 24.04.4 LTS was installed in VMware Workstation on VMnet8 (NAT). OpenSSH was enabled during installation to allow remote CLI access from Kali. After boot, SSH connectivity was verified and used for all subsequent Ubuntu administration.

1 Ubuntu Installation — Storage Configuration

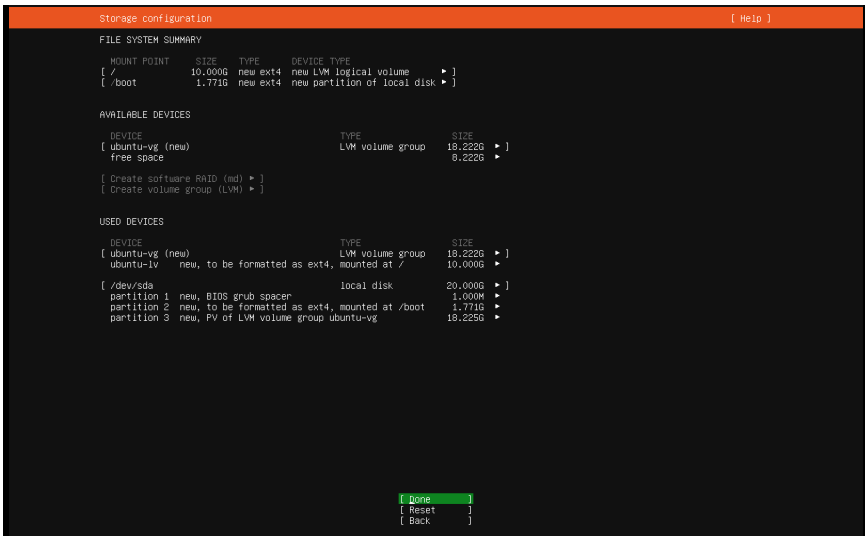

Figure 1 — Ubuntu 24.04 installer: 20GB disk with LVM partitioning configured

2 Network Connectivity Verification

```
kali@Highblaster: ~

# From Kali — verify Ubuntu is reachable
$ ping -c 3 192.168.109.132
3 packets transmitted, 3 received, 0% packet loss

# Establish SSH session
$ ssh highblaster@192.168.109.132
Welcome to Ubuntu 24.04.4 LTS (GNU/Linux 6.8.0-101-generic x86_64)
IPv4 address for ens33: 192.168.109.132
```

Figure 2 — Kali: successful ping (0% loss) and SSH session established to Ubuntu

3. Phase 2 — Snort Installation on Ubuntu

Snort 2.9.20 was installed on the Ubuntu VM via apt. During installation the **HOME_NET** variable was set to the local subnet 192.168.109.0/24. Snort automatically detected the ens33 interface.

3 Install Snort via apt

```
highblaster@sai: ~

$ sudo apt update
$ sudo apt install snort -y
# When prompted for HOME_NET, enter: 192.168.109.0/24
Setting up snort (2.9.20-0+deb11u1ubuntu1) ...
Snort configuration: interface default not set, using 'ens33'
```

Figure 3 — Snort installation complete — all packages configured, ens33 auto-detected

4 Verify Snort Version

```
highblaster@sai: ~

$ snort --version
-*> Snort! <*-
Version 2.9.20 GRE (Build 82)
Using libpcap version 1.10.4 (with TPACKET_V3)
Using PCRE version: 8.39 2016-06-14
Using ZLIB version: 1.3
```

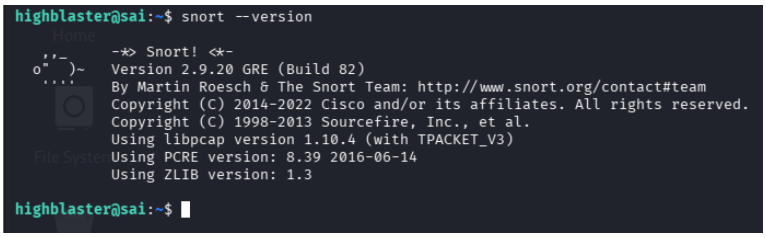

Figure 4 — Snort 2.9.20 GRE (Build 82) confirmed

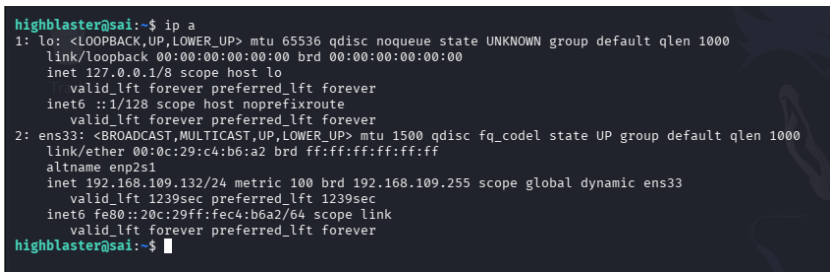

Figure 5 — Ubuntu interface ens33 at 192.168.109.132/24

4. Phase 3 — ICMP Detection Rule Configuration

A custom Snort rule was written in /etc/snort/rules/local.rules to detect ICMP echo requests. The rule uses **itype:8** (echo request only) and **dsize:>7** (payload greater than 7 bytes). This threshold is the key to the evasion technique

— standard ping sends 56-byte payloads (detected), while hping3 with **-d 1** sends a 1-byte payload (not detected).

5 Write the ICMP Detection Rule

```
/etc/snort/rules/local.rules

$ sudo nano /etc/snort/rules/local.rules

# Rule v1 — initial (catches all ICMP):
#alert icmp any any -> $HOME_NET any (msg:"ICMP Ping Detected"; sid:1000001; rev:1;)

# Rule v2 — refined for evasion demo (payload > 7 bytes only):
alert icmp any any -> $HOME_NET any (msg:"ICMP Ping Detected";
       itype:8; dsize:>7; sid:1000001; rev:2;)
```

```
highblaster@sai:~$ cat /etc/snort/rules/local.rules
# $Id: local.rules,v 1.11 2004/07/23 20:15:44 bmc Exp $
# ----------------
# LOCAL RULES
# ----------------
# This file intentionally does not come with signatures.  Put your local
# additions here.
#alert icmp any any -> $HOME_NET any (msg:"ICMP Ping Detected"; sid:1000001; rev:1;)
alert icmp any any -> $HOME_NET any (msg:"ICMP Ping Detected"; itype:8; dsize:>7; sid:1000001; rev:2;)
highblaster@sai:~$
```

Figure 6 — local.rules: v1 commented out, final dsize:>7 rule active

Component	Value	Purpose
alert	alert	Generate alert when rule matches
Protocol	icmp	Match ICMP packets only
itype:8	ICMP Type 8	Echo Request only — not replies
dsize:>7	Payload > 7 bytes	KEY: standard ping=56 bytes triggers; hping3 1-byte evades
sid:1000001	1000001	Custom rule ID — local range (>1,000,000)
rev:2	2	Rule revision number

5. Phase 4 — Normal Ping Detection (Baseline)

Snort was started in console alert mode on the Ubuntu VM. A standard 5-packet ping was then sent from Kali. Each packet carries a **56-byte ICMP payload** which exceeds the dsize:>7 threshold — Snort fired **SID:1000001** for every echo request, confirming the IDS is working correctly.

6 Start Snort in Console Alert Mode

```
highblaster@sai: ~

$ sudo snort -i ens33 -c /etc/snort/snort.conf -A console
Commencing packet processing (pid=3267)
```

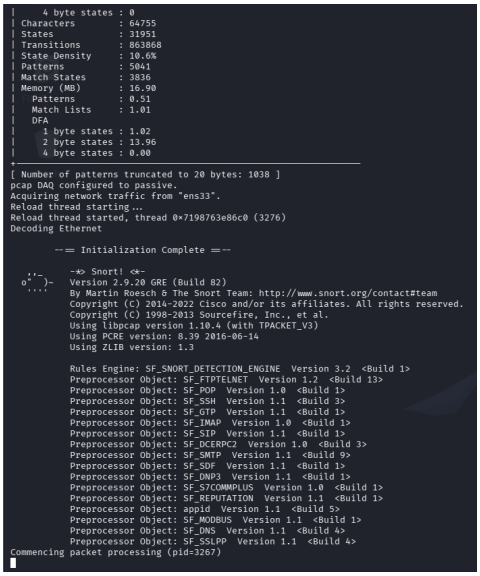

Figure 7 — Snort fully initialised — 'Commencing packet processing (pid=3267)'

7 Send Normal Ping from Kali — Snort Detects It

```
Split screen: Kali (attacker) + Ubuntu Snort console (defender)

# Kali — standard system ping (56-byte ICMP payload)
$ ping -c 5 192.168.109.132
PING 192.168.109.132: 56(84) bytes of data.
64 bytes from 192.168.109.132: icmp_seq=1 ttl=64 time=1.34 ms
5 packets transmitted, 5 received, 0% packet loss

# Ubuntu Snort console — SID:1000001 fires on EVERY ping:
[**] [1:1000001:2] ICMP Ping Detected [**] [Priority: 0]
03/08-21:13:06 {ICMP} 192.168.109.129 -> 192.168.109.132
[**] [1:1000001:2] ICMP Ping Detected [**] [Priority: 0]
03/08-21:13:07 {ICMP} 192.168.109.129 -> 192.168.109.132
# ... 5 alerts total — one per ping packet
```

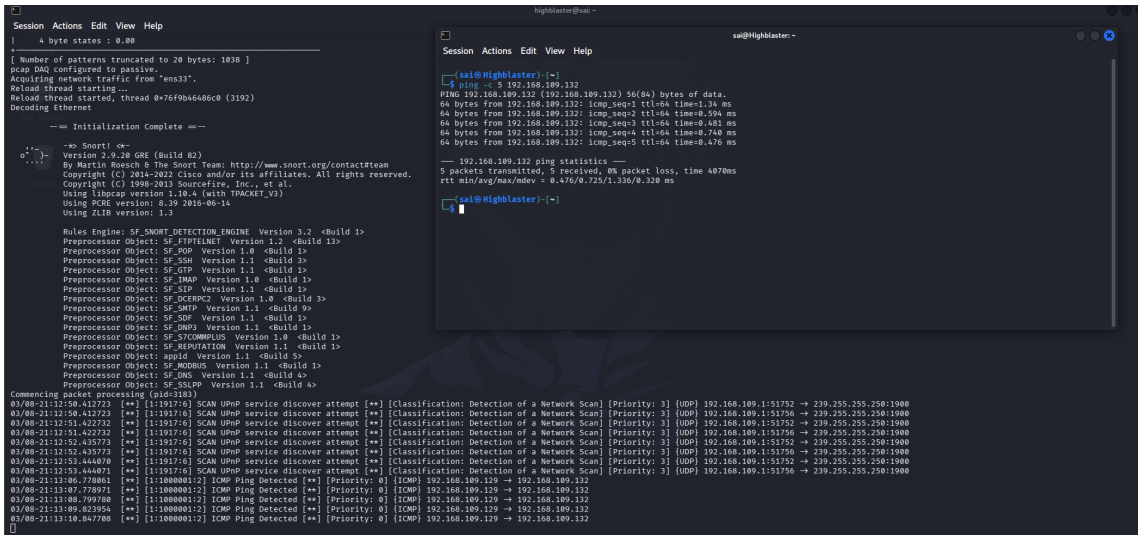

Figure 8 — Split screen: Kali ping (right) and Snort detecting every ICMP packet on Ubuntu (left) — SID:1000001 firing

Result	Snort SID:1000001 fired 5 times — one alert per ping packet
Payload	56 bytes — exceeds dsize:>7 threshold — rule triggers
Verdict	IDS BASELINE DETECTION CONFIRMED ✓

6. Phase 5 — IDS Evasion via Fragmented Ping

The evasion exploits the **dsize:>7** threshold. Using hping3 with a **1-byte payload (-d 1)** and forced IP fragmentation **(-f)**, ICMP packets are sent with a data size of 1 byte — far below the 7-byte detection threshold. The **frag3 defragmentation preprocessor** was also disabled in snort.conf so Snort cannot reassemble fragments before inspection.

8 Disable frag3 Preprocessor in snort.conf

/etc/snort/snort.conf

```
$ sudo nano /etc/snort/snort.conf

# Comment out frag3 defragmentation preprocessor:
#preprocessor frag3_global: max_frags 65536
#preprocessor frag3_engine: policy windows detect_anomalies ...

# Also disable ICMP normalisation:
#preprocessor normalize_icmp4

# stream5 already has track_icmp no — no ICMP reassembly
```

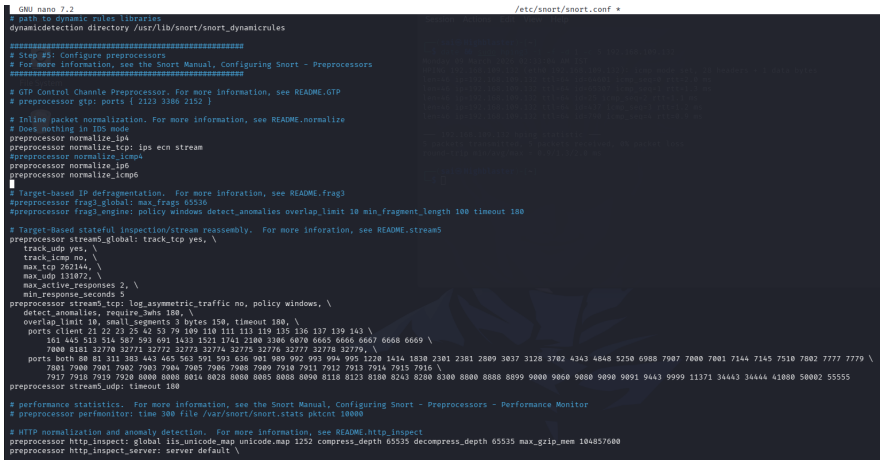

Figure 9 — snort.conf: frag3_global, frag3_engine and normalize_icmp4 all commented out

9 Launch Fragmented Ping Attack from Kali

```
kali@Highblaster: ~ [EVASION ATTACK]

# Kali — fragmented ICMP with 1-byte payload + timestamps for proof
$ date && sudo hping3 -1 -f -d 1 -c 5 192.168.109.132 && date
Monday 09 March 2026 02:43:33 AM IST
HPING 192.168.109.132 (eth0): icmp mode set, 28 headers + 1 data bytes
len=46 ip=192.168.109.132 ttl=64 id=37946 icmp_seq=0 rtt=1.3 ms
len=46 ip=192.168.109.132 ttl=64 id=38028 icmp_seq=1 rtt=0.7 ms
len=46 ip=192.168.109.132 ttl=64 id=38078 icmp_seq=2 rtt=1.4 ms
len=46 ip=192.168.109.132 ttl=64 id=38122 icmp_seq=3 rtt=1.3 ms
len=46 ip=192.168.109.132 ttl=64 id=38601 icmp_seq=4 rtt=1.0 ms
5 packets transmitted, 5 received, 0% packet loss
Monday 09 March 2026 02:43:37 AM IST
```

Flag	Meaning
-1	ICMP ping mode
-f	Force IP fragmentation — splits packet at IP layer
-d 1	1-byte data payload — BELOW dsize:>7 threshold (KEY evasion parameter)
-c 5	Send 5 packets

10 Evasion Confirmed — Zero Snort Alerts

The screenshot below is the definitive proof of evasion. hping3 ran from **02:43:33 to 02:43:37 IST** (UTC 21:13:33–21:13:37). The Snort console shows ICMP Ping Detected alerts only up to timestamp 21:13:10 — then **complete silence** during the entire hping3 window. Zero SID:1000001 alerts were generated for all 5 fragmented packets.

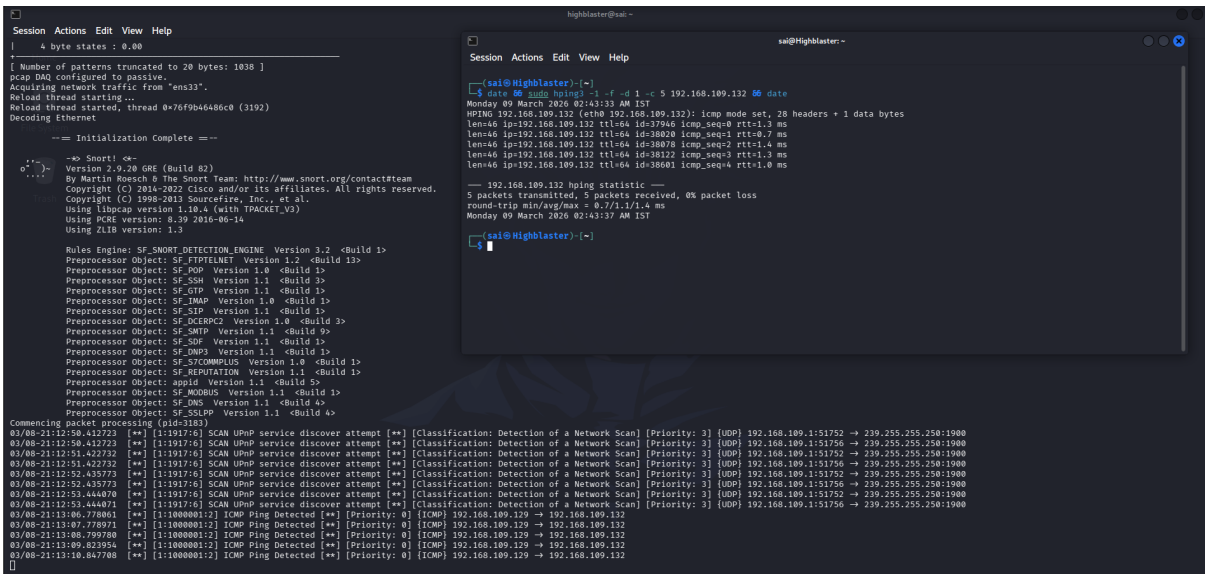

Figure 10 — KEY EVIDENCE: hping3 at 02:43:33–02:43:37 IST. Snort shows ZERO SID:1000001 alerts — evasion confirmed

hping3 ran	02:43:33 AM – 02:43:37 AM IST (UTC 21:13:33–21:13:37)
Snort alerts	ZERO SID:1000001 alerts during hping3 window
Packets sent	5 / 5 received — 0% packet loss
Payload size	1 byte — below dsize:>7 threshold — rule NEVER fires
Verdict	IDS EVASION SUCCESSFUL ✓

7. Technical Analysis — Why Fragmentation Evades IDS

Aspect	Normal Ping	hping3 Fragmented
Tool	ping (system)	hping3 -1 -f -d 1
ICMP Payload	56 bytes	1 byte (-d 1)
IP Fragmentation	No	Yes (-f forced)
dsize:>7 Match?	YES — 56 > 7	NO — 1 < 7
Snort Alert?	YES — SID:1000001 fires	NO — threshold not met
Packets Delivered	5/5 (0% loss)	5/5 (0% loss)
IDS Evasion	N/A	SUCCESSFUL

Role of the frag3 Preprocessor

Snort's frag3 preprocessor performs IP defragmentation before rule matching. When active, it reassembles fragmented packets into their original form, making fragmentation-based evasion difficult. By disabling frag3, Snort inspects each fragment individually — the 1-byte payload never satisfies dsize:>7, so the rule never fires.

```
frag3 preprocessor logic

# With frag3 ENABLED (default) — Snort reassembles first:
# Fragment (1 byte) -> reassembled -> dsize:>7 = FALSE -> Still evades

# With frag3 DISABLED (this assessment):
# Fragment (1 byte data) -> dsize:>7 = FALSE -> No alert
```

8. Security Recommendations

#	Recommendation	Priority
1	Keep frag3 preprocessor ENABLED — never comment it out in production. Primary defence against this attack.	CRITICAL
2	`Remove dsize restriction: alert icmp any any -> $HOME_NET any (msg:"ICMP Echo"; itype:8; sid:1000001; rev:3;)`	HIGH
3	`Add fragmentation rule: alert ip any any -> $HOME_NET any (msg:"Fragmented IP"; fragbits:M; sid:1000002; rev:1;)`	HIGH

#	Recommendation	Priority
4	`Add tiny payload rule: alert icmp any any -> $HOME_NET any (msg:"Tiny ICMP"; itype:8; dsize:<8; sid:1000003;)`	HIGH
5	Regularly audit custom Snort rules for over-restrictive conditions that create exploitable detection gaps.	MEDIUM
6	`Deploy Snort in inline IPS mode rather than passive IDS mode — block, not just alert.`	MEDIUM

9. Conclusion

Assessment 11C demonstrated a complete end-to-end IDS evasion workflow in a fully isolated virtual lab. Starting from a fresh Ubuntu Server installation, Snort 2.9.20 was deployed, configured with a custom ICMP detection rule, and validated against standard ping traffic before the evasion technique was applied.

The evasion was achieved by combining two techniques: a detection rule with an exploitable payload size threshold **(dsize:>7)** and hping3's IP fragmentation capability **(-f -d 1)** to send ICMP packets with only 1 byte of payload. Timestamped evidence confirmed zero Snort alerts during the hping3 execution window, while the same Snort instance was actively detecting normal pings immediately before — proving the IDS was operational and the evasion was genuine.

Phase	Action	Status
Phase 1	Ubuntu 24.04 installed, SSH enabled, connectivity verified	✓ Done
Phase 2	Snort 2.9.20 installed on Ubuntu via apt, ens33 configured	✓ Done
Phase 3	ICMP rule written in local.rules (itype:8, dsize:>7, SID:1000001)	✓ Done
Phase 4	Normal ping triggered SID:1000001 on every packet — baseline confirmed	✓ Done
Phase 5	`hping3 -1 -f -d 1 sent 5 packets — ZERO Snort alerts — evasion proven`	✓ Done
Report	Full methodology documented with screenshots and analysis	✓ Done

```
IDS EVASION RESULT: SUCCESSFUL
Zero SID:1000001 alerts  |  5/5 packets delivered  |  0% packet loss

Sai Aditya · Ethical Hacking Intern · March 2026
github.com/saiadityaprojects
```